

Ассемблер

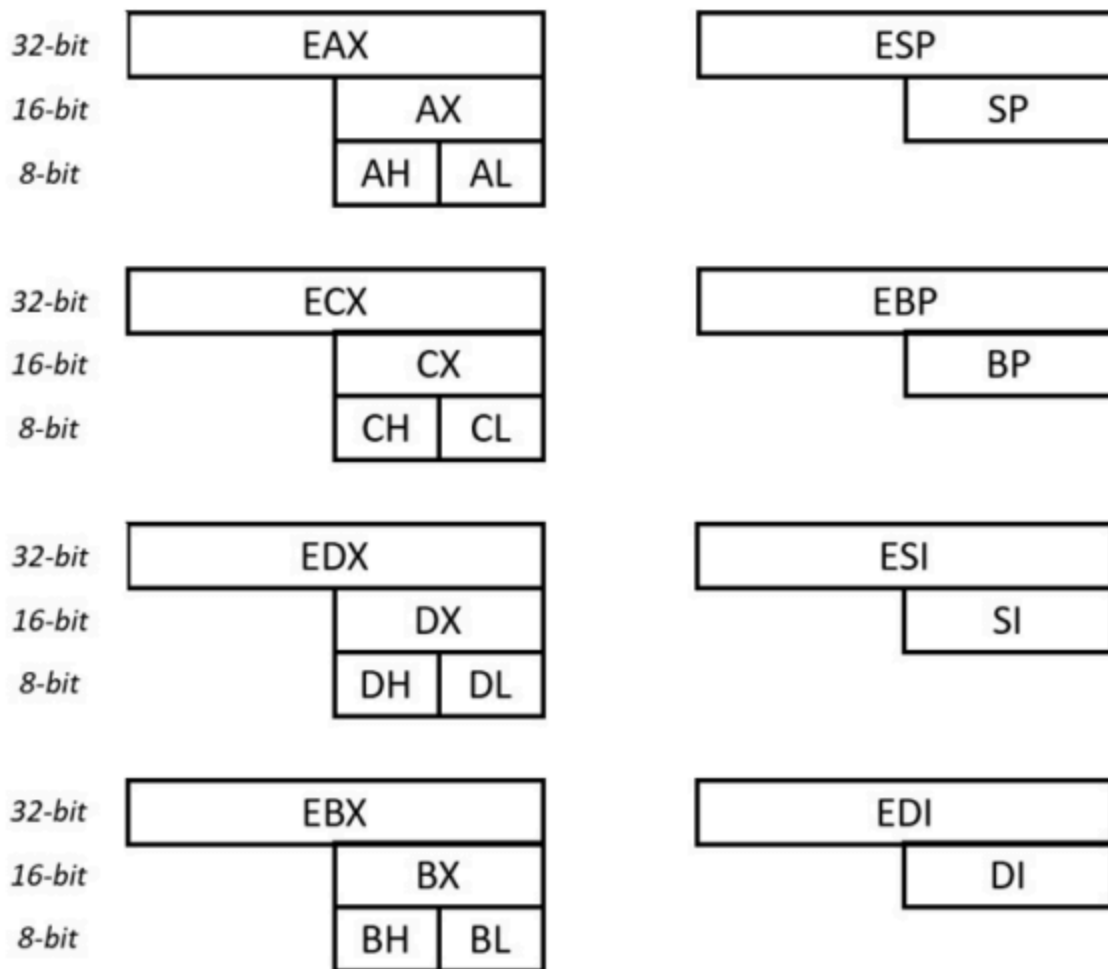
Знакомство с ассемблером будет происходить концептуально, т.е. непосредственного программирования не будет.

Ассемблер — представление команд процессора в виде, доступном для чтения человеком. Программы, написанные на языке ассемблера, однозначным образом переводятся в инструкции конкретного процессора и в большинстве случаев не могут быть перенесены без значительных изменений для запуска на машине с другой системой команд.

Программирование на ассемблере условно можно представлять следующим образом: Есть некоторый набор ящиков. Можно что-то взять из какого-то ящика, можно что-то положить в ящик. Также есть некоторые индикаторы, которые говорят о результате различных действий или операций.

Ящики

В качестве "ящиков" служат регистры процессора. Они играют ключевую роль в обработке данных в процессоре. Регистры в процессоре x86-64 можно разделить на четыре категории: регистры общего назначения, специальные регистры для приложений, сегментные регистры и специальные регистры режима ядра. В основном используются **регистры общего назначения**.

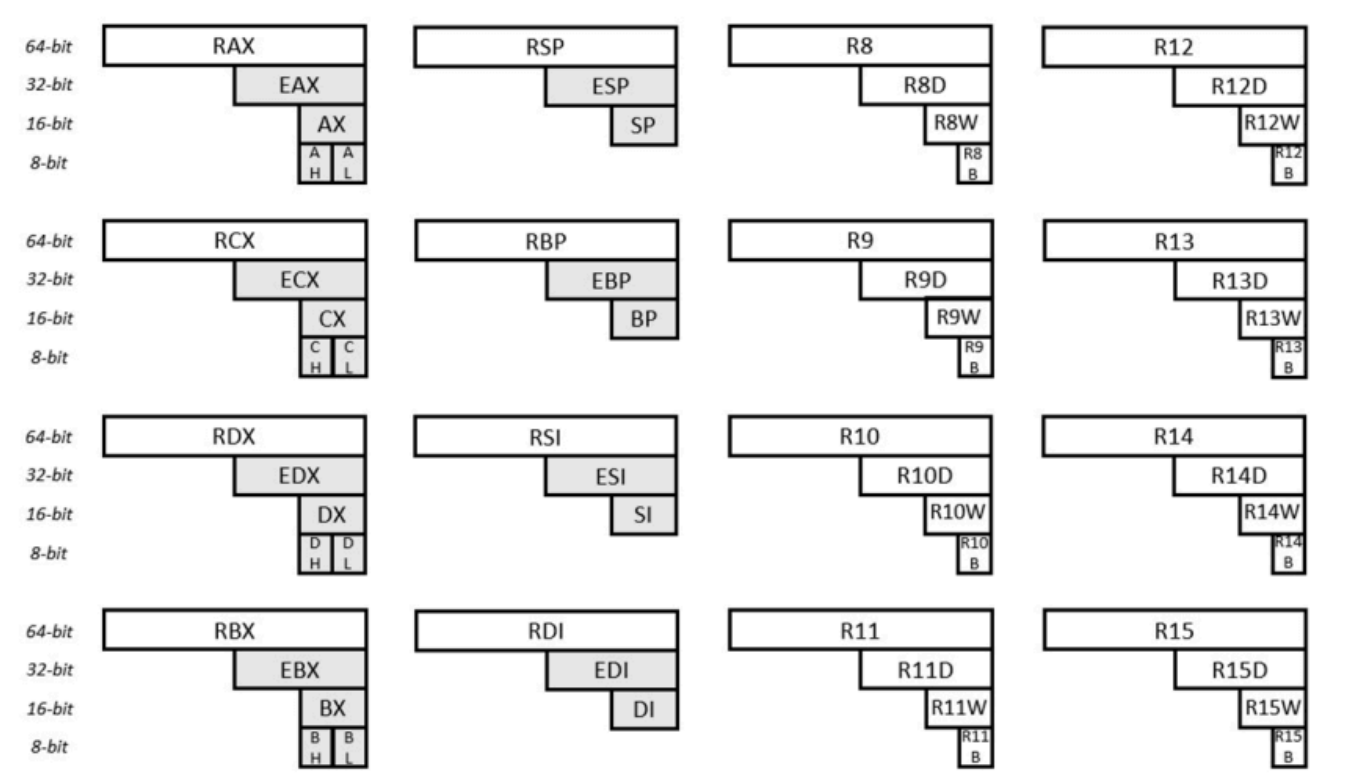


Процессор архитектуры x86 имел восемь 32-битных регистров общего назначения, регистр флагов и указатель инструкций. Регистры общего назначения:

- **EAX** (Accumulator): для арифметических операций;
- **ECX** (Counter): для хранения счетчика цикла;
- **EDX** (Data): для арифметических операций и операций ввода-вывода;
- **EBX** (Base): указатель на данные;
- **ESP** (Stack pointer): указатель на верхушку стека;
- **EBP** (Base pointer): указатель на базу стека внутри функции;
- **ESI** (Source index): указатель на источник при операциях с массивом;
- **EDI** (Destination index): указатель на место назначения в операциях с массивами;
- **EIP**: указатель адреса следующей инструкции для выполнения;
- **EFLAGS**: регистр флагов, содержит биты состояния процессора;

Также можно получить доступ к частям 32-битных регистров с меньшей разрядностью. Например, младшие 16 бит 32-битного регистра EAX обозначаются как AX. К регистру AX можно обращаться как к отдельным байтам, используя имена AH (старший байт) и AL (младший байт).

В архитектуре x64 эти регистры были расширены до 64 бит, а новые расширенные регистры получили имена, которые начинаются с буквы R, например, RAX, RBX и т.д. Кроме того, были добавлены 8 новых 64-битных регистров R8 - R15



Флаги

Регистр флагов RFLAGS, содержит биты состояния процессора:

Бит	Имя	Назначение
0	CF	Флаг переноса (Carry flag): показывает, был ли при сложении перенос или заимствование при вычитании. Используется в качестве входных данных для инструкций сложения и вычитания.
2	PF	Флаг четности: устанавливается, если младшие 8 битов результата содержат четное число единиц.
4	AF	Флаг настройки: указывает, был ли при сложении перенос или заимствование при вычитании младших 4 битов.
6	ZF	Флаг нуля (Zero flag): устанавливается, если результат операции равен нулю.
7	SF	Флаг знака (Sign flag): устанавливается, если результат операции отрицательный.
8	TF	Флаг прерывания выполнения (Trap flag): используется при одношаговой отладке.

Бит	Имя	Назначение
9	IF	Флаг разрешения прерывания: установка этого бита разрешает аппаратные прерывания.
10	DF	Флаг направления: контролирует направления обработки. Если не установлен, то порядок от самого младшего до самого старшего адреса. Если установлен, то порядок обратный.
11	OF	Флаг переполнения (Overflow flag): если установлен, то операция привела к переполнению со знаком.
12-13	IOPL	Уровень привилегий ввода-вывода (I/O privilege level): уровень привилегий текущего выполняемого потока. IOPL 0 — это режим ядра, а 3 — пользовательский режим.
14	NT	Флаг вложенной задачи (Nested task flag): управляет цепочкой прерываний.
16	RF	Флаг возобновления (Resume flag): используется для обработки исключений во время отладки.
17	VM	Флаг режима виртуальной машины 8086: если установлен, режим совместимости с 8086 активен. Этот режим позволяет запускать некоторые приложения MS-DOS в контексте операционной системы в защищенном режиме.
18	AC	Флаг проверки выравнивания (Alignment check flag): если установлен, то проверка выравнивания памяти активна. Например, если установлен флаг AC, сохранение 16-битного значения по нечетному адресу вызывает исключение проверки выравнивания. Процессоры x86 могут выполнять невыровненный доступ к памяти, когда этот флаг не установлен, но количество требуемых командных циклов может увеличиться.
19	VIF	Флаг виртуального прерывания (Virtual interrupt flag): виртуальная версия флага IF в виртуальном режиме 8086.
20	VIP	Флаг ожидания виртуального прерывания: Устанавливается, когда прерывание находится в состоянии ожидания в виртуальном режиме x8086.
21	ID	Флаг ID: если этот бит установлен, то поддерживается инструкция cpushd. Эта инструкция возвращает идентификатор процессора и информацию о его функциях.

Основными являются флаги переноса, нуля, знака и переполнения, которые называют флагами состояния.

Условные переходы

Также в ассемблере есть ряд инструкций, которые выполняют условные переходы в зависимости от установки битов в регистре флагов. В частности, учитываются четыре бита: **carry** (флаг переноса), **overflow** (флаг переполнения), **sign** (флаг знака) и **zero** (флаг нуля).

Эти флаги устанавливаются в результате различных операций. В частности, инструкции **add**, **sub**, **and**, **or**, **xor** и **not** влияют на установку флагов.

Для проверки этих флагов и выполнения условного перехода в ассемблере есть следующие инструкции:

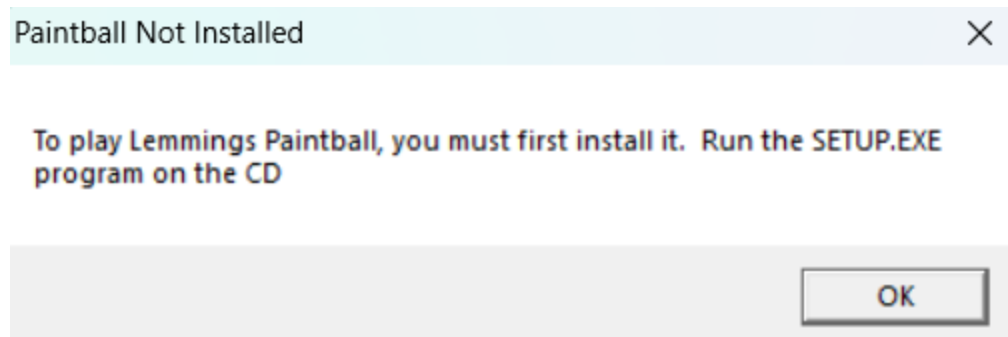
- **jc**: выполняет переход к метке, если флаг переноса установлен;
- **jnc**: выполняет переход к метке, если флаг переноса **НЕ** установлен;
- **jo**: выполняет переход к метке, если флаг переполнения установлен;
- **jno**: выполняет переход к метке, если флаг переполнения **НЕ** установлен;
- **js**: выполняет переход к метке, если флаг знака установлен;
- **jns**: выполняет переход к метке, если флаг переполнения **НЕ** установлен;
- **jz**: выполняет переход к метке, если флаг нуля установлен;
- **jnz**: выполняет переход к метке, если флаг **НЕ** установлен;

Reverse Engineering

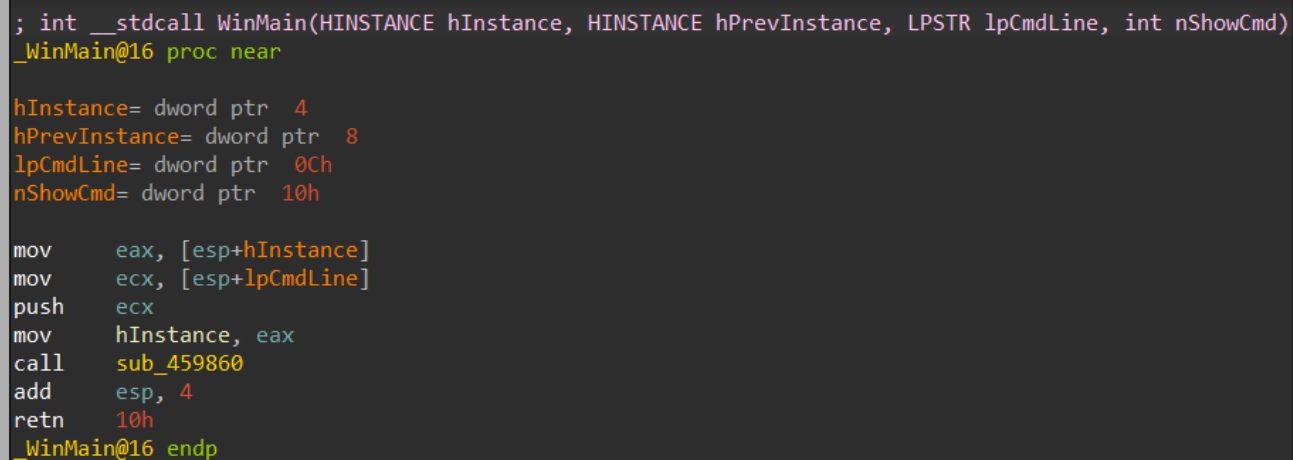
Знакомство с реверсом будет проходить путем решения задачи [Lemballovo](#). Для решения нам понадобится дизассемблер IDA (или любой другой наверное тоже подойдет).

	DSPDIB16.DLL	01.06.2020 5:03	Расширение при...	4 КБ
	DSPDIB32.DLL	01.06.2020 5:03	Расширение при...	15 КБ
	LEMBALLEXE	01.06.2020 5:03	Приложение	721 КБ
	PBAIMOG.VSR	01.06.2020 5:03	Файл "VSR"	9 655 КБ

Открываем EXE-шник и видим ошибку:



Открываем EXE-шник в IDA и ничего не понимаем:

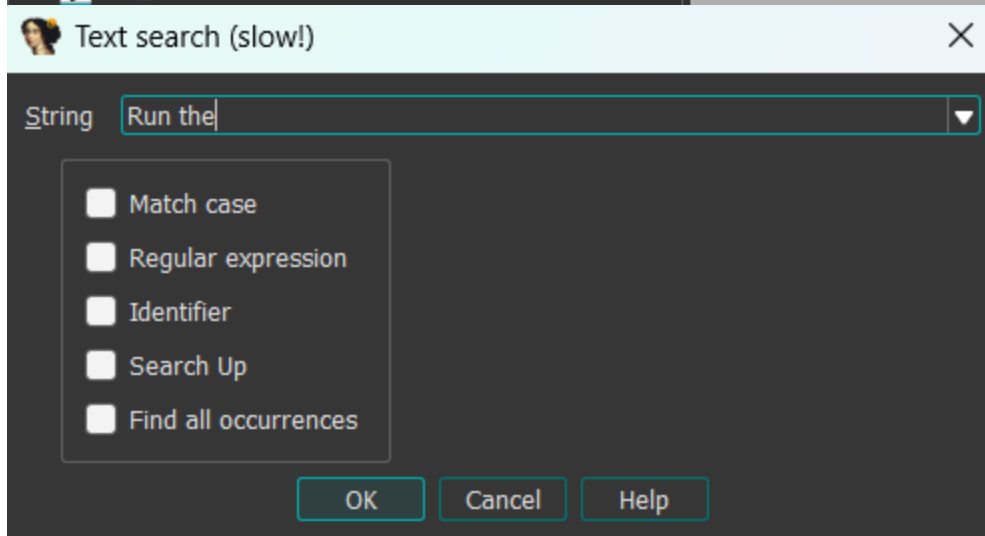
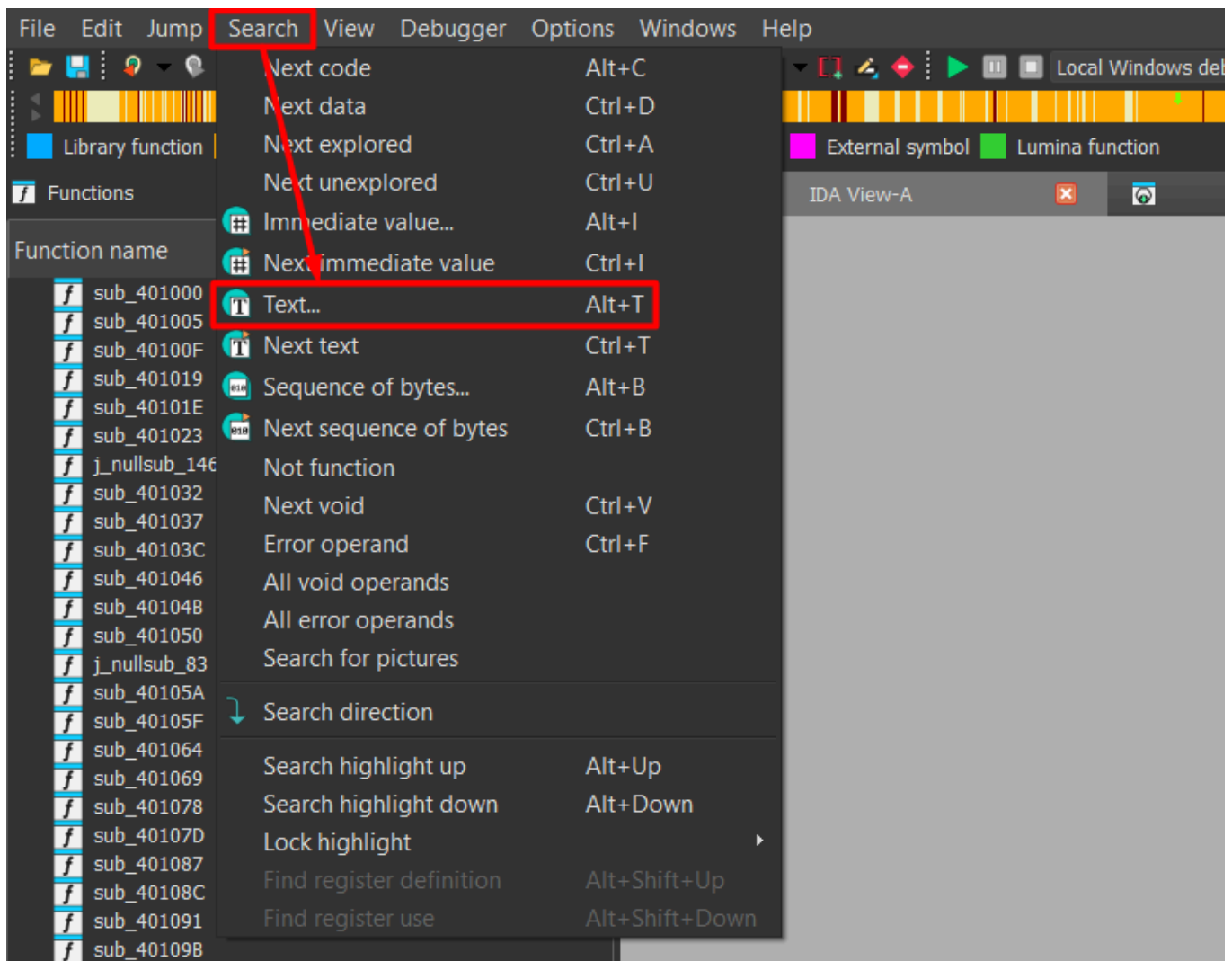
A screenshot of the IDA Pro disassembler window. The assembly code for the WinMain function is displayed. The code includes parameter declarations, stack frame setup, and a call to a sub_459860 function.

```
; int __stdcall WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd)
_WinMain@16 proc near

hInstance= dword ptr  4
hPrevInstance= dword ptr  8
lpCmdLine= dword ptr  0Ch
nShowCmd= dword ptr  10h

mov     eax, [esp+hInstance]
mov     ecx, [esp+lpCmdLine]
push    ecx
mov     hInstance, eax
call    sub_459860
add     esp, 4
retn    10h
_WinMain@16 endp
```

Но у нас есть зацепка — сообщение об ошибке. Нужно воспользоваться поиском строк в IDA:



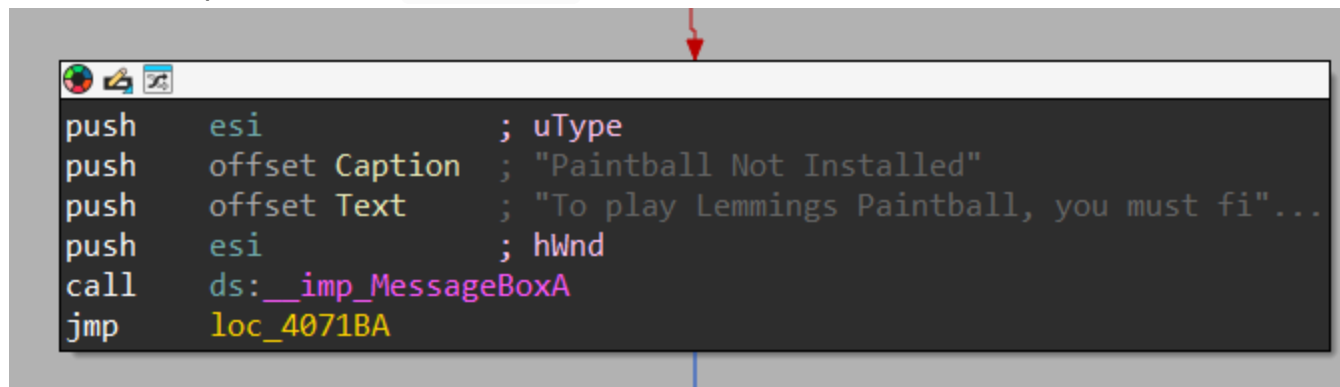
Заботливая IDA находит нам наше сообщение об ошибке:

```
.data:0049CBF4 ; CHAR Text[]  
.data:0049CBF4 Text          db 'To play Lemmings Paintball, you must first install it. Run the S'  
.data:0049CBF4              ; DATA XREF: sub_406DF0+501o  
.data:0049CC35              db 'ETUP.EXE program on the CD',0
```

Также IDA показывает нам, где используется это сообщение:

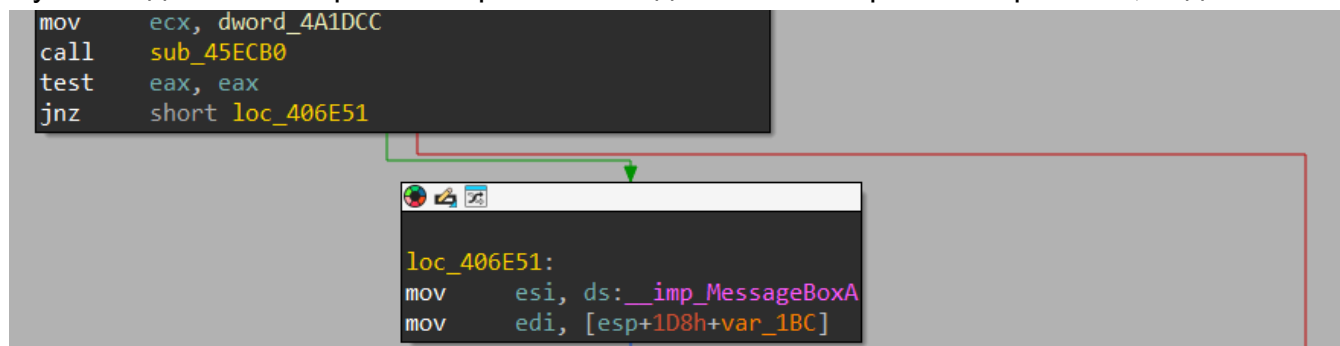
```
; DATA XREF: sub_406DF0+5010
```

Нажимаем 2 раза ЛКМ по `sub_406DF0` . Видим:



```
push    esi           ; uType
push    offset Caption ; "Paintball Not Installed"
push    offset Text    ; "To play Lemmings Paintball, you must fi"...
push    esi           ; hWnd
call    ds:__imp_MessageBoxA
jmp     loc_4071BA
```

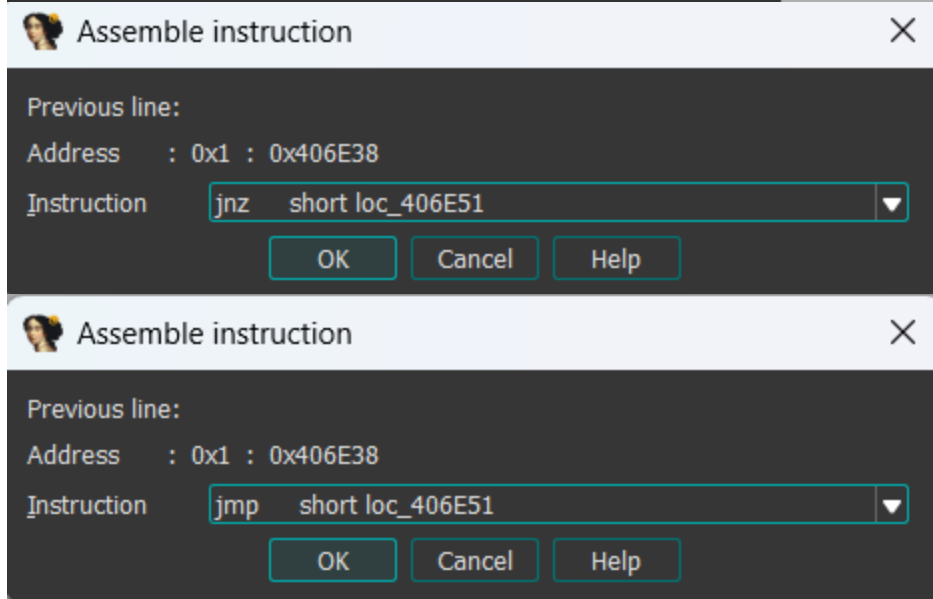
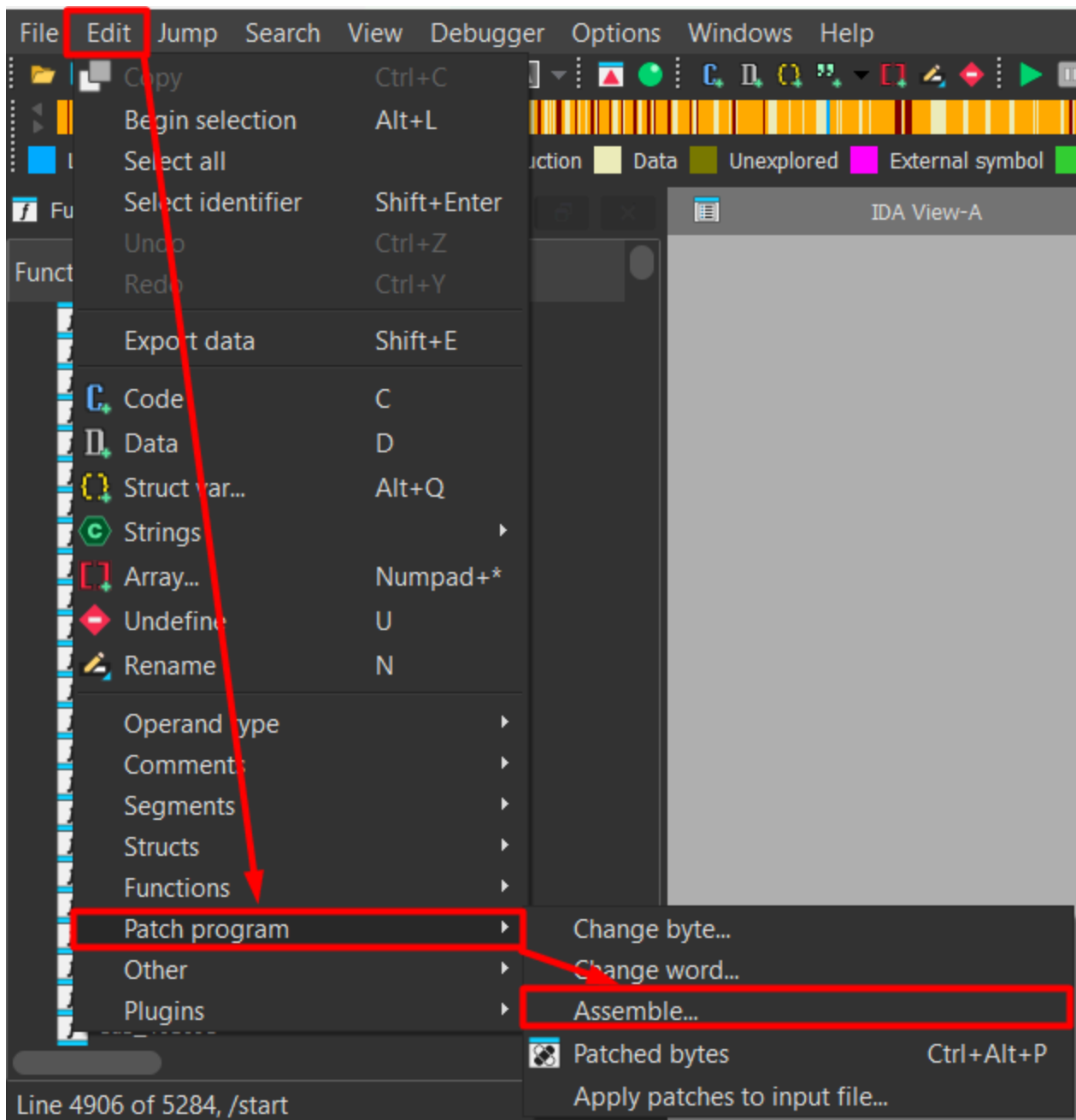
Нужно подняться по красной стрелочке. Поднявшись по красной стрелочке, видим:



```
mov     ecx, dword_4A1DCC
call    sub_45ECB0
test    eax, eax
jnz     short loc_406E51
```

```
loc_406E51:
mov     esi, ds:__imp_MessageBoxA
mov     edi, [esp+1D8h+var_1BC]
```

В верхнем фрагменте видно, что происходит вызов какой-то функции, затем выполняется условный переход `jnz` на `loc_406E51` (зелёная стрелочка). Очевидно, что, т.к. мы получаем сообщение об ошибке, то в `ZF` всё время 0. Нужно поменять `jnz` на `jmp` (или на `jb`). Для этого ставим курсор на `jnz` , а дальше делаем как на скриншотах ниже:



Нажимаем ОК, затем на крестик (или на Cancel). После этого видим, что изменилась инструкция:

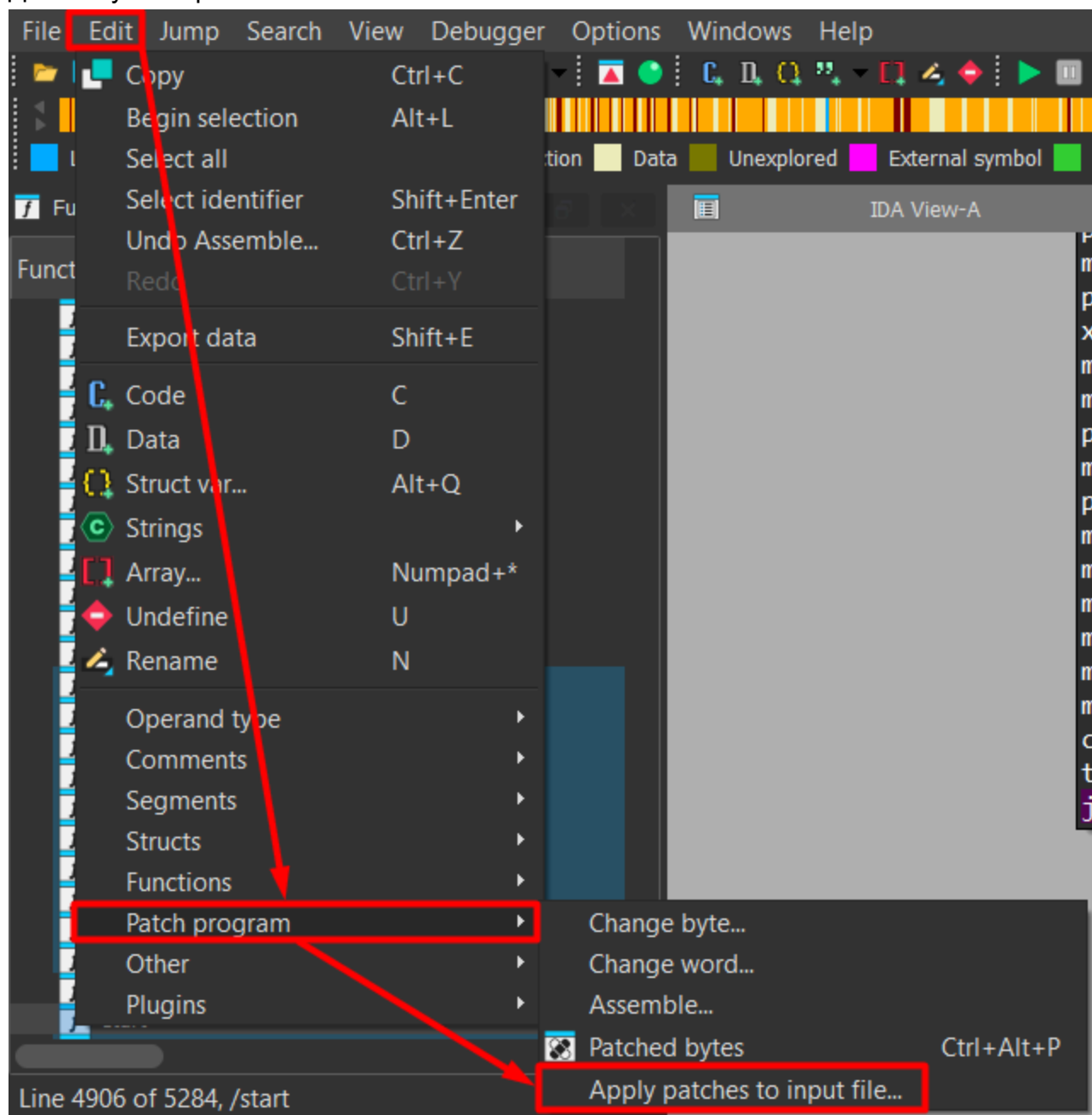
```
call    sub_45ECB0
test    eax, eax
jmp     short loc_406E51
```

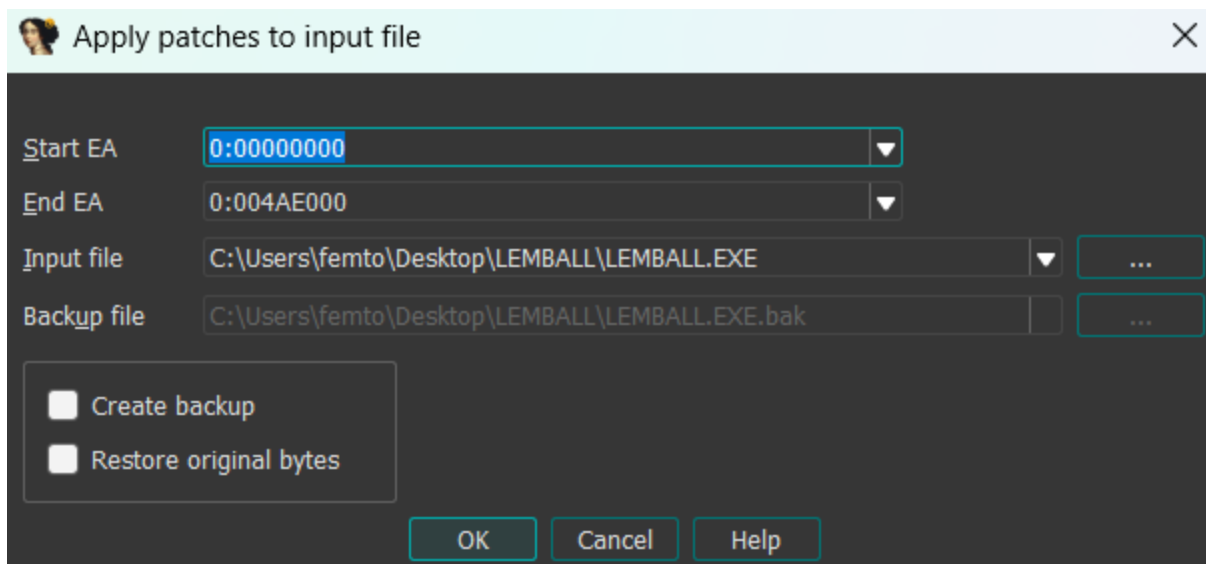


```
loc_406E51:
mov     esi, ds:__imp_MessageBoxA
mov     edi, [esp+1D8h+var_1BC]
```

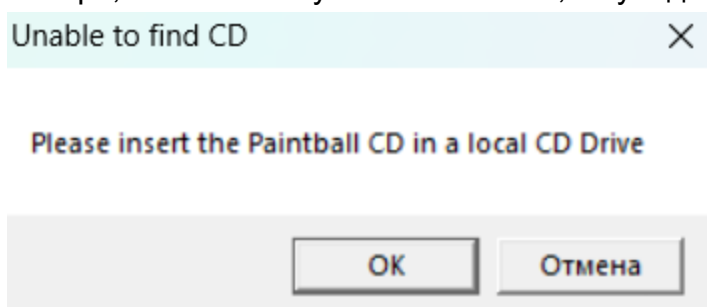
Пропала стрелочка к окну с ошибкой.

Далее нужно применить наш патч.





Теперь, если мы запустим EXE-шник, то увидим другое сообщение об ошибке:

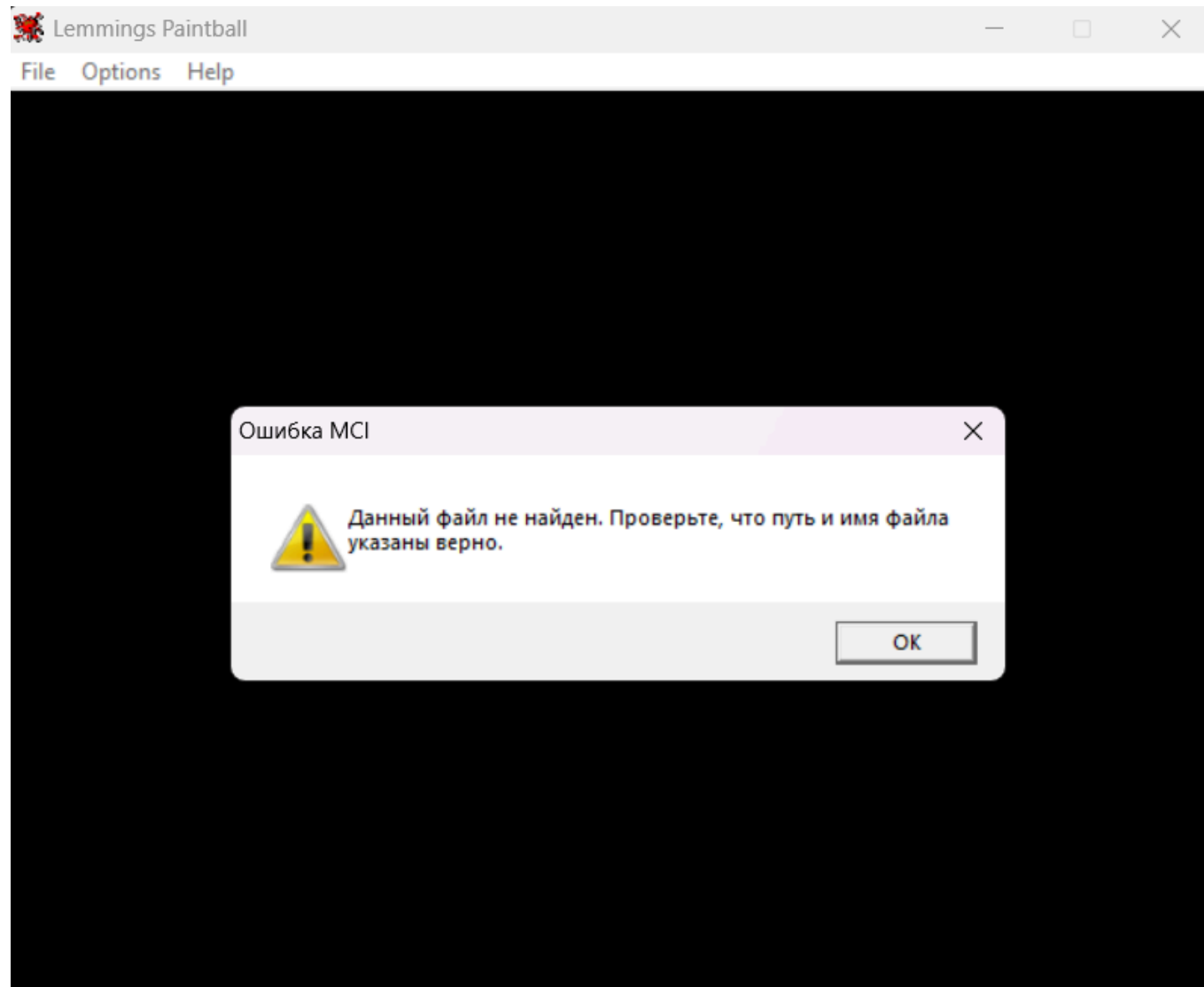


Таким же способом находим нужный кусок кода и патчим его. Повторяем так до тех пор, пока не победим.

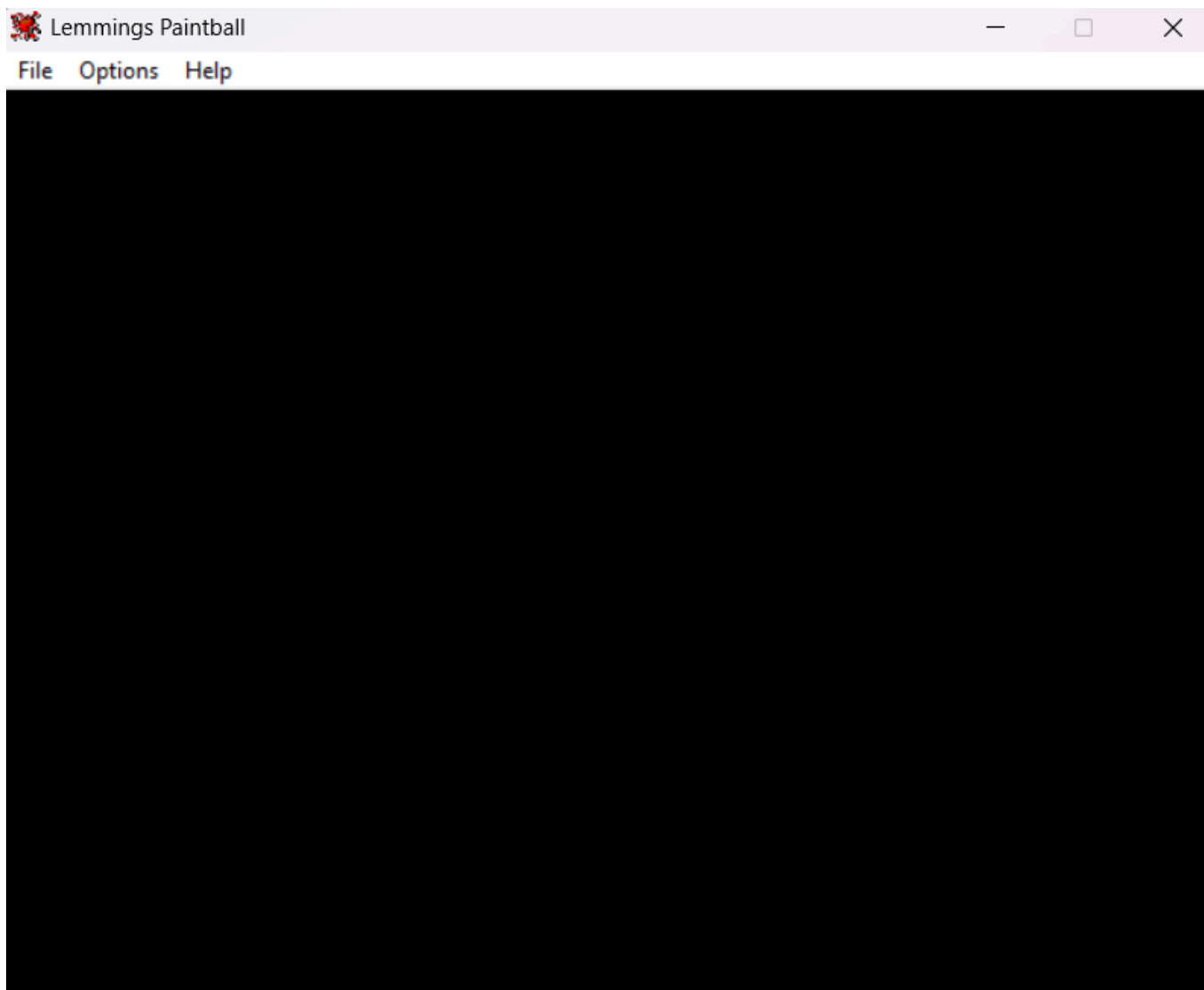
Запатченные байты:

Address	Length	Original bytes	Patched bytes
00406E38	0x1	75	EB
00406E6D	0x1	75	EB
0043A3D7	0x1	74	EB

После всех патчей, если запустить EXE-шник, то будет вот такая шляпа:



Жмём ОК



Затем один раз ЛКМ по черному фону (вроде как можно и не жать, можно чуть-чуть

подождать) и получаем нашу игру. Флаг ищите сами)

